

ui.carousel 全面详解

ui.carousel 是 NiceGUI 基于 Quasar 的 QCarousel 组件封装的轮播图元素，用于循环展示多个幻灯片（`ui.carousel_slide`）内容，支持动画切换、手动导航、自动播放等核心功能，广泛适用于图片展示、产品介绍、内容轮播等场景。其核心特性包括灵活的导航配置、幻灯片切换控制、双向绑定等，可快速实现交互友好的轮播效果。

一、核心概念与基础特性

1. 本质与用途

- 本质：封装 Quasar 的 QCarousel 组件，通过嵌套 `ui.carousel_slide` 子元素定义单个轮播幻灯片，每个幻灯片可包含图片、文本、按钮等任意 NiceGUI 元素。
- 核心用途：在有限空间内循环展示多个内容块，提升信息展示效率与页面交互性，常见于首页 Banner、产品图库、公告轮播等场景。
- 关键机制：支持手动导航（箭头、圆点指示器）和程序控制（切换上一张 / 下一张），幻灯片切换可配置动画效果，默认不启用动画与导航控件，需手动开启。

2. 基础结构

一个完整的 ui.carousel 由轮播容器（`ui.carousel()`）、幻灯片（`ui.carousel_slide()`）和幻灯片内容（如图片、文本）组成，示例结构如下：

```
from nicegui import ui

# 轮播容器：启用动画、箭头导航、圆点导航，设置高度
with ui.carousel(animated=True, arrows=True, navigation=True).props('height=300px') as carousel:
    # 幻灯片1：展示图片
    with ui.carousel_slide().classes('p-0'):
        ui.image('https://picsum.photos/id/237/800/300').classes('w-full h-full object-cover')
    # 幻灯片2：展示文本+按钮
    with ui.carousel_slide().classes('flex items-center justify-center'):
        ui.column(
            ui.label('第二张幻灯片').classes('text-2xl font-bold'),
            ui.button('点击查看', on_click=lambda: ui.notify('点击了第二张幻灯片'))
        )
    # 幻灯片3：展示复杂布局
    with ui.carousel_slide().classes('p-4'):
        ui.row(
            ui.image('https://picsum.photos/id/10/400/300').classes('rounded'),
            ui.column(
                ui.label('第三张幻灯片标题').classes('text-xl'),
                ui.label('幻灯片描述文本：支持多行内容与复杂布局'),
                classes='m1-4'
            )
        )
```

```
ui.run()
```

二、初始化配置项

初始化 `ui.carousel()` 时可通过参数配置核心行为与样式，参数说明如下：

参数名	类型	说明
value	ui.carousel_slide 实例或幻灯片名称字符串	初始选中的幻灯片（默认值为 None，即选中第一张幻灯片）
on_value_change	Callable	幻灯片切换时触发的回调函数（接收 <code>valueChangeEventArguments</code> 参数，包含当前选中幻灯片信息）
animated	bool	是否启用幻灯片切换动画（默认 False，设为 True 时切换有过渡效果）
arrows	bool	是否显示箭头导航按钮（默认 False，设为 True 时显示左右箭头用于手动切换）
navigation	bool	是否显示圆点导航指示器（默认 False，设为 True 时底部显示圆点，点击可跳转对应幻灯片）

配置示例

```
# 初始选中“slide2”，切换时触发回调，启用动画、箭头和圆点导航
def on_slide_change(e):
    # e.value 为当前选中的幻灯片实例
    ui.notify(f'当前幻灯片: {e.value.name if e.value.name else "未命名"}')

with ui.carousel(
    value='slide2',
    on_value_change=on_slide_change,
    animated=True,
    arrows=True,
    navigation=True
).props('height=200px') as carousel:
    with ui.carousel_slide(name='slide1'):
        ui.label('第一张幻灯片').classes('text-center')
    with ui.carousel_slide(name='slide2'):
        ui.label('第二张幻灯片').classes('text-center')
    with ui.carousel_slide(name='slide3'):
        ui.label('第三张幻灯片').classes('text-center')
```

三、核心属性

`ui.carousel` 继承 NiceGUI 基础元素的通用属性，支持样式、类名、绑定等配置，关键属性如下：

属性名	类型	说明
classes	str	元素的 CSS 类名（支持 Tailwind、Quasar 类，如 <code>w-full</code> 占满宽度、 <code>mx-auto</code> 水平居中）
props	str	Quasar 组件属性（用于扩展功能，如 <code>autoplay=3000</code> 启用自动播放，间隔 3 秒； <code>loop</code> 启用循环播放）
style	str	内联 CSS 样式（如 <code>border-radius: 8px;</code> 设置圆角）
value	BindableProperty	当前选中的幻灯片（可通过 <code>bind_value</code> 绑定到外部变量，支持双向同步）
visible	BindableProperty	元素可见性（布尔值，支持动态绑定）
html_id	str	HTML DOM 中的元素 ID（版本 2.16.0 新增，用于精准定位）
is_deleted	bool	元素是否已被删除（只读属性）
parent_slot	Slot None	父容器的插槽（可手动设置元素所属父插槽）

属性使用示例

```
# 占满宽度、圆角边框、自动播放（3秒切换）、循环播放的轮播图
with ui.carousel(
    animated=True,
    arrows=True,
    navigation=True
).props('height=250px autoplay=3000 loop').classes('w-full rounded-lg border-2 border-gray-200'):
    ui.carousel_slide().classes('p-0')
    ui.image('https://picsum.photos/id/20/800/250').classes('w-full h-full object-cover')
    ui.carousel_slide().classes('p-0')
    ui.image('https://picsum.photos/id/21/800/250').classes('w-full h-full object-cover')
```

四、核心方法

ui.carousel 提供丰富的方法用于控制幻灯片切换、元素管理、绑定等操作，常用方法分类如下：

1. 幻灯片控制方法

方法名	作用	示例
next()	切换到下一张幻灯片	<code>carousel.next()</code>
previous()	切换到上一张幻灯片	<code>carousel.previous()</code>
set_value(value)	设置当前选中幻灯片（值为 ui.carousel_slide 实例、幻灯片名称或索引）	<code>carousel.set_value('slide3')</code> 或 <code>carousel.set_value(2)</code>

2. 元素管理方法

方法名	作用	参数说明
clear()	删除轮播图内所有幻灯片	无参数
remove(element)	删除指定幻灯片	element: 幻灯片实例 (ui.carousel_slide 对象) 或其 ID
delete()	删除整个轮播图元素及所有子元素	无参数
move(target_container=None, target_index=-1, target_slot=None)	移动轮播图到其他容器	target_container: 目标容器 (默认父容器) ; target_index: 目标索引 (默认追加到末尾) ; target_slot: 目标插槽

3. 绑定方法

支持将幻灯片选中状态、可见性与外部变量绑定，支持单向 / 双向绑定，核心方法如下：

方法名	作用	关键参数
bind_value(target_object, target_name='value')	双向绑定选中幻灯片到目标对象的属性	target_object: 绑定目标对象; target_name: 绑定的属性名 (默认 'value')
bind_value_from(...)	单向绑定 (从目标对象同步到轮播图)	同 bind_value, 仅单向同步 (目标对象属性变化触发轮播图选中状态变化)
bind_value_to(...)	单向绑定 (从轮播图同步到目标对象)	同 bind_value, 仅单向同步 (轮播图选中状态变化触发目标对象属性变化)
bind_visibility(...)	双向绑定可见性到目标对象的属性	value: 可选, 指定目标值匹配时才显示轮播图

4. 其他常用方法

方法名	作用	示例
tooltip(text)	为轮播图添加鼠标悬浮提示	<code>carousel.tooltip('产品图片轮播')</code>
update()	强制更新客户端的轮播图状态 (如动态添加幻灯片后刷新)	<code>carousel.update()</code>

方法名	作用	示例
add_resource(path)	为轮播图添加资源文件（如自定义 CSS、JS）	<code>carousel.add_resource('./static')</code>
ancestors(include_self=False)	迭代获取所有祖先元素	遍历祖先元素： <code>for elem in carousel.ancestors(): print(elem)</code>
mark(*markers)	为元素添加标记（用于测试或元素查询）	<code>carousel.mark('product-carousel', '2024')</code>

方法使用示例

```
from nicegui import ui

# 程序控制轮播图切换
def go_to_prev():
    carousel.previous()

def go_to_next():
    carousel.next()

def go_to_first():
    carousel.set_value(0) # 通过索引设置（0为第一张）

with ui.carousel(animated=True).props('height=200px') as carousel:
    with ui.carousel_slide():
        ui.label('第一张').classes('text-center text-xl')
    with ui.carousel_slide():
        ui.label('第二张').classes('text-center text-xl')
    with ui.carousel_slide():
        ui.label('第三张').classes('text-center text-xl')

# 自定义控制按钮
ui.row(
    ui.button('上一张', on_click=go_to_prev),
    ui.button('下一张', on_click=go_to_next),
    ui.button('回到第一张', on_click=go_to_first)
).classes('mt-4')

ui.run()
```

五、动态幻灯片管理

ui.carousel 支持动态添加、删除或调整幻灯片位置，核心依赖 `ui.carousel_slide` 实例与轮播图的元素管理方法，典型场景如下：

场景 1：动态添加幻灯片

```
from nicegui import ui

slides = []

def add_slide():
    # 新增幻灯片，添加到轮播图末尾
    with carousel:
        slide = ui.carousel_slide(name=f'slide{len(slides)+1}')
        ui.label(f'动态添加的幻灯片 {len(slides)+1}').classes('text-center text-xl')
        slides.append(slide)
        carousel.update() # 刷新轮播图

with ui.carousel(animated=True, arrows=True, navigation=True).props('height=200px') as carousel:
    # 初始幻灯片
    with ui.carousel_slide(name='slide1'):
        ui.label('初始幻灯片1').classes('text-center text-xl')
        slides.append(carousel.default_slot.children[0])

# 添加按钮
ui.button('添加幻灯片', on_click=add_slide).classes('mt-4')

ui.run()
```

场景 2：动态删除幻灯片

```
def delete_last_slide():
    if slides:
        slide_to_delete = slides.pop()
        carousel.remove(slide_to_delete) # 删除指定幻灯片
        carousel.update() # 刷新轮播图
        ui.notify(f'删除了幻灯片: {slide_to_delete.name}')

# 在上述示例中添加删除按钮
ui.button('删除最后一张', on_click=delete_last_slide).classes('ml-2')
```

场景 3：调整幻灯片位置

```
def move_slide_to_first(index):
    if 0 <= index < len(slides):
        slide = slides.pop(index)
        slide.move(target_index=0) # 移动到第一张位置
        slides.insert(0, slide)
        carousel.update()
        ui.notify(f'已将幻灯片 {slide.name} 移到第一张')

# 在上述示例中添加位置调整按钮
ui.button('将最后一张移到第一张', on_click=lambda:
move_slide_to_first(len(slides)-1)).classes('ml-2')
```

六、事件处理

1. 核心事件：幻灯片切换事件

通过 `on_value_change` 配置切换回调，获取当前选中幻灯片信息：

```
def handle_slide_change(e):
    slide_index = carousel.default_slot.children.index(e.value)
    ui.notify(f'幻灯片切换到第 {slide_index+1} 张（名称：{e.value.name}）')

with ui.carousel(on_value_change=handle_slide_change, animated=True) as carousel:
    # 幻灯片定义...
```

2. 通用事件绑定

通过 `on()` 方法绑定 DOM 事件（如点击、鼠标悬浮），支持 Python 或 JavaScript 回调：

```
# 轮播图容器点击事件（Python 回调）
carousel.on('click', lambda e: ui.notify('点击了轮播图容器'))

# 幻灯片点击事件（为单个幻灯片绑定）
slide = ui.carousel_slide()
slide.on('click', lambda: ui.notify('点击了当前幻灯片'))
```

七、高级用法

1. 样式深度自定义

结合 `props`、`classes` 和 `style` 实现个性化样式，示例：

```
# 自定义箭头颜色、圆点颜色、轮播图边框
with ui.carousel(
    animated=True,
    arrows=True,
    navigation=True
```

```

).props(
    'height=250px autoplay=4000 loop',
    'color=primary', # 箭头和圆点颜色
    'arrow-size=24px', # 箭头大小
    'navigation-size=12px' # 圆点大小
).classes('w-full max-w-3xl mx-auto rounded-xl border-4 border-blue-500 overflow-hidden'):
    ui.carousel_slide().classes('p-0')
    ui.image('https://picsum.photos/id/30/800/250').classes('w-full h-full object-cover')
    ui.carousel_slide().classes('p-0')
    ui.image('https://picsum.photos/id/31/800/250').classes('w-full h-full object-cover')

```

2. 双向绑定选中状态

将轮播图选中状态与外部变量绑定，实现状态同步：

```

from nicegui import ui

class CarouselState:
    current_slide = None

state = CarouselState()

with ui.carousel(animated=True) as carousel:
    # 绑定选中状态到 state.current_slide
    carousel.bind_value(state, 'current_slide')
    with ui.carousel_slide(name='slide1'):
        ui.label('幻灯片1')
    with ui.carousel_slide(name='slide2'):
        ui.label('幻灯片2')

# 外部显示当前选中状态
ui.label('当前选中: ').bind_text_from(state, 'current_slide', lambda s: s.name if s else '无')

# 外部控制选中状态
ui.button('选中幻灯片2', on_click=lambda: setattr(state, 'current_slide',
    carousel.default_slot.children[1]))

ui.run()

```

3. 嵌套复杂交互元素

幻灯片内可嵌套表单、图表、弹窗等复杂交互元素，示例：

```

from nicegui import ui

with ui.carousel(animated=True, arrows=True).props('height=300px'):
    # 幻灯片内嵌套表单
    with ui.carousel_slide().classes('p-6'):
        ui.label('表单幻灯片').classes('text-xl font-bold mb-4')
        with ui.card():
            ui.input('姓名', placeholder='请输入姓名')
            ui.input('邮箱', placeholder='请输入邮箱')

```



```
        ui.button('提交', on_click=lambda: ui.notify('表单提交成功')).classes('mt-2')
# 幻灯片内嵌套图表
with ui.carousel_slide().classes('p-6'):
    ui.label('图表幻灯片').classes('text-xl font-bold mb-4')
    ui.chart({
        'type': 'bar',
        'data': {'labels': ['A', 'B', 'C'], 'datasets': [{'data': [10, 20, 15]}]}
    }).classes('w-full h-[200px]')
```

八、注意事项

1. 尺寸控制：轮播图默认高度较窄，建议通过 `props('height=xxxpx')` 或 `style` 明确设置高度，避免内容溢出或显示异常。
2. 图片适配：幻灯片内图片建议添加 `object-cover` 类（Tailwind），确保图片按比例填充幻灯片区域，避免拉伸变形。
3. 自动播放：需通过 Quasar props `autoplay=毫秒数` 启用自动播放（如 `autoplay=3000` 表示 3 秒切换一次），结合 `loop` props 实现循环播放。
4. 版本兼容性：`html_id` 属性需 NiceGUI 2.16.0+ 版本支持，`bind_value` 的 `strict` 参数需 3.0.0+ 版本支持，使用时需确认版本匹配。
5. 动态更新：动态添加、删除或调整幻灯片后，需调用 `update()` 方法刷新客户端显示，确保轮播图状态同步。
6. 事件冲突：避免在幻灯片内绑定与轮播图导航（如箭头点击）冲突的事件，防止交互异常。

通过以上配置与方法，`ui.carousel` 可灵活满足从简单图片轮播到复杂交互轮播的各类需求，是 NiceGUI 中实现内容循环展示的核心组件之一。